Duta | Engineering Change Specification
مستند بازطراحی معماری فرانت‌اند، بک‌اند و Nginx برای مقیاس ۱,۰۰۰,۰۰۰ کاربر و حداقل مصرف RAM/CPU
CHG-009

مشخصات سند

نسخه	V1.0
تاریخ	1405/07/30
تیم	Duta
وضعیت	تأیید شده

تاریخچه تغییرات

تحلیلگر	عنوان یا توضیحات	تاریخ
مانی مردآزاد ناوی	بازطراحی معماری کشینگ چندلایه، تردهای مجازی Java 21 ، استخر Keepalive در Nginx و حذف گلوگاه‌های فرانت‌اند	1405/07/30

عنوان تغییر

شناسه	CHG-009
عنوان	بهینه‌سازی حداکثری پرفورمنس، حذف کوئری‌های N+1، کاهش مصرف RAM/CPU و آماده‌سازی زیرساخت برای ۱ میلیون کاربر
لایه سیستم	Full-Stack Architecture (Frontend React 19, Backend Java 21/Spring Boot 3.3, Nginx Reverse Proxy)
اولویت	بحرانی(Critical)

توصیف

این سند شامل کلیه اصلاحات معماری و مهندسی نرم‌افزار در سه لایه فرانت‌اند، بک‌اند و وب‌سرور Nginx است که بدون ایجاد هیچ‌گونه تغییر در ظاهر (UI/UX) یا منطق تجاری برنامه، گلوگاه‌های پردازشی و شبکه‌ای را حذف کرده و سیستم را برای پاسخ‌گویی پایدار به ترافیک وایرال (تا ۱,۰۰۰,۰۰۰ کاربر) روی سرور ابری با حداقل مصرف RAM و CPU آماده می‌سازد.

جدول تفصیلی تغییرات و دلایل فنی — بخش اول: معماری فرانت‌اند (Frontend Architecture)

کد مورد	بخش / فایل	وضعیت قبلی (مشکل فعلی)	ریسک و مشکل در آینده	وضعیت جدید (تغییر اعمال‌شده)	دلیل فنی تصمیم‌گیری
FE-ARCH-01	main.jsx و tabler-icons-subset.css	ایمپورت کل پکیج @tabler/icons-webfont (شامل ۱۳۰ کیلوبایت CSS برای ۵۴۰۰ آیکون + دانلود فایل فونت سنگین woff2).	ایجاد تاخیر ۱۴۳۰ میلی‌ثانیه‌ای در رندر موبایل (Font display) و خطای Reduce unused CSS (21 KiB در Lighthouse).	ساخت فایل بومی ۲ کیلوبایتی tabler-icons-subset.css با استفاده از mask-image برداری دقیقاً برای ۲۸ آیکون مورد استفاده در Duta با همان کلاس‌های ti ti-*.	حفظ ۱۰۰٪ ظاهر آیکون‌ها در کنار حذف کامل درخواست فونت خارجی و کاهش ۹۸٪ حجم CSS آیکون‌ها.
FE-ARCH-02	PriceTicker.jsx	Mount شدن همزمان دو نمونه از PriceTicker (یکی در هدر موبایل و یکی در Navbar دسکتاپ).	ارسال ۲ درخواست همزمان GET /api/prices در بدو ورود هر کاربر و اجرای ۲ تایمر ۵ دقیقه‌ای موازی در حافظه مرورگر.	پیاده‌سازی الگوی **Singleton Module Store** (sharedPrices, subscribers, activePromise) در PriceTicker.jsx.	اشتراک‌گذاری یک درخواست شبکه و یک وضعیت واحد بین هدر موبایل و دسکتاپ (کاهش ۵۰٪ ترافیک اندپوینت قیمت).
FE-ARCH-03	src/utils/articleHelpers.js و کارت‌های خبر	تعریف مجدد توابع formatTime، getBadgeClass	در هر بار رندر شدن لیست	استخراج تمامی توابع محاسباتی و فرمت زمان به ماژول خالص (Pure	حذف کامل تخصیص

		و getScoreLabel در داخل بدنه کامپوننت‌های NewsCard، TelegramNewsCard و ArticleDetail.	اخبار، صدها Closure تکراری در حافظه ساخته و توسط Garbage Collector پاک می‌شد.	(Functions در src/utils/articleHelpers.js.	حافظه تکراری در رندرها (Zero Allocation per Card Render) و یکپارچگی ۱۰۰٪ کدها.
FE-ARCH-04	App.css ، NewsGrid.jsx و Home.jsx	۱ .انیمیشن Shimmer با تغییر background-position روی CPU. ۲ .تخریب و بازسازی تایمر ۵ دقیقه‌ای با هر اسکرول. ۳ .تگ‌های <main> تودرتو.	۱ .درگیر شدن Main Thread پردازنده موبایل (2.9s) و اخطار Avoid non-composited animations. ۲ .ریست شدن تایمر بررسی اخبار جدید هنگام صفحه‌بندی .	۱ .انتقال انیمیشن Shimmer به کارت گرافیک (GPU) با transform: translateX روی .shimmer::after. ۲ .تثبیت تایمر با useRef و کنترل resize با requestAnimationFrame. ۳ .تبدیل <main> های داخلی به .<section>	رساندن بار گرافیکی اسکلتون روی CPU به ۰ میلی‌ثانیه، اسکرول ۶۰ فریم بر ثانیه و ساختار معنایی استاندارد HTML5.

جدول تفصیلی تغییرات و دلایل فنی — بخش دوم: معماری بک‌اند (Backend 1M-User Scalability)

کد مورد	بخش / فایل	وضعیت قبلی (مشکل فعلی)	ریسک و مشکل در آینده	وضعیت جدید (تغییر اعمال‌شده)	دلیل فنی تصمیم گیری
BE-SCALE-01	CacheConfig.java و ArticleService.java	کش شدن صرفاً لیست صفحه اول و بدون کش بودنِ /slug/{slug}، /clusters/{id}/articles/seo/{slug} و /sitemap.xml.	در زمان وایرال شدن یک خبر فوری (مثلاً ۱۰۰,۰۰۰ کلیک در ۵ دقیقه)، ۲۰۰,۰۰۰ کوئری مستقیم به دیتابیس زده می‌شد و تولید مکرر رشته ۷۸۵ کیلوبایتی /sitemap.xml رم سرور را پر می‌کرد.	تعریف ۶ ناحیه کش مجزا در Caffeine (articles, article_detail, clusters, seo, sitemap, categories) با سقف ۲۰۰۰ ورودی و ابطال هم‌زمان (@Caching) در زمان درج خبر جدید.	پاسخ‌های 0ms از روی RAM به ۱,۰۰۰,۰۰۰ کاربر هم‌زمان در اخبار وایرال بدون تحمیل حتی ۱ کوئری اضافی به دیتابیس.
BE-SCALE-02	ArticleRepository.java	واکشی تک‌خبر و مقالات کلستر بدون JOIN FETCH روی روابط source و cluster.	اجرای کوئری‌های اضافی N+1 توسط	بازنویسی کوئری‌های findBySlug، findByIdWithDetails و findByClusterIdWithSources با دستور LEFT JOIN FETCH a.source LEFT JOIN FETCH a.cluster.	کاهش تعداد کوئری های دیتابیس از

			Hibernate برای پر کردن اطلاعات منبع و کلستر هر خبر.		2N + 1به ۱ کوئری واحد در زمان Cache Miss.
BE-SCALE-03	RssFetcherService.java	اجرای کوئری مستقیم articleRepository.existsByUrl(link) به ازای تک‌تک آیتم‌های داخل فیدهای RSS در هر ۵ دقیقه.	ارسال حدود ۹۰۰ کوئری تکراری SELECTدر هر ۵ دقیقه برای لینک‌هایی که ۹۵٪ آن‌ها در سیکل قبلی بررسی شده بودند.	افزودن کش درون‌حافظه‌ای seenUrlsCache) با ظرفیت ۵۰۰۰ لینک و TTL ۱۲ ساعته در (Caffeine پیش از کوئری دیتابیس.	حذف ۹۵٪ کوئری های دیتابیس در سیکل‌های ۵ دقیقه‌ای RSS با مصرف کمتر از ۱ مگابایت RAM.
BE-SCALE-04	ClusteringService.java و ScoringService.java	۱ .توکنایز کردن مجدد تیتر تمامی کلسترها با Regex به ازای هر خبر جدید. ۲ .ساخت مجموعه‌های HashSetجدید در هر محاسبه Jaccard. ۳ .واکشی دوبل findByCluster.	ایجاد ۱۵,۰۰۰ عملیات Regex و هزاران شیء موقت در حافظه Heap در هر سیکل RSS)فشار	۱ .افزودن tokenCacheبرای ذخیره توکن‌های هر تیتر. ۲ .محاسبه ریاضی اشتراک و اجتماع جاکارد بدون ساخت HashSetجدید (Zero-Allocation). ۳ .پاس دادن مستقیم لیست مقالات به ScoringService.	کاهش مصرف CPU و تخصیص حافظه (Heap Allocation (در الگوری

				شدید به CPUو GC).	تم خوشه بندی به بیش از ۹۰٪.
BE-SCALE-05	RateLimitConfig.java	نگهداری دو ConcurrentHashMapدستی (requestCountsو windowStartبا پاکسازی دستی.	احتمال بروز Race Conditionدر ریست پنجره زمانی و رشد حافظه در حملات توزیع‌ش ده (DDoS).	بازنویسی کامل با Caffeine.newBuilder().maximumSize(10_000).expireAfterWrite(1, MINUTES).	مدیریت اتمیک، Lock-freeو خودکار حافظه با الگوری تم WindowTinyLFU برای ده هزار آی‌پی فعال در هر دقیقه.
BE-SCALE-06	application*.propertiesو pom.xml	۱ .استفاده از تردهای سنتی سیستم‌عامل (Platform Threads). ۲ .وجود کتابخانه بلااستفاده org.telegram:telegrambots:6.8.0.	۱ . محدودیت تامکت به ۲۰۰ درخواست همزمان و مصرف ۱ مگابایت رم به ازای هر	۱ .فعال‌سازی **Java 21 Virtual Threads** (spring.threads.virtual.enabled=true). ۲ .حذف پکیج بلااستفاده telegrambotsاز pom.xml.	**توانایی مدیریت ده‌ها هزار کانکشن هم‌زمان با حداقل مصرف RAM**

			ترد. ۲ . تداخل comm ons- loggin g.jarدر لاگ استارت اپ و ۳۰ مگابایت حجم اضافی JAR.		**و حذف کامل اخطار commons-loggingدر استارتاپ.**

جدول تفصیلی تغییرات و دلایل فنی — بخش سوم: معماری Nginx و شبکه (Infrastructure Layer)

کد مورد	بخش / فایل	وضعیت قبلی (مشکل فعلی)	ریسک و مشکل در آینده	وضعیت جدید (تغییر اعمال‌شده)	دلیل فنی تصمیم‌گیری
INF-SCALE-01	nginx.conf ($client_real_ip)	ارسال مستقیم $remote_addr در هدر X-Real-IP در حالی که سایت پشت CDN پارس‌پک (WCDN) قرار دارد.	چون $remote_addr آی‌پی سرور لبه CDN پارس‌پک است، تمام کاربران سایت با یک آی‌پی واحد به بک‌اند معرفی می‌شدند و با رسیدن مجموع ترافیک کل سایت به ۱۲۰ درخواست در دقیقه، **همه کاربران با خطای 429 مسدود می‌شدند!**	استخراج اولین آی‌پی واقعی کاربر از زنجیره $http_x_forwarded_for با استفاده از بلاک map ($client_real_ip) و ارسال آن در X-Real-IP.	تفکیک دقیق آی‌پی تک‌تک کاربران واقعی پشت CDN پارس‌پک و جلوگیری از مسدودسازی دسته‌جمعی کاربران.
INF-SCALE-02	nginx.conf (upstream keepalive)	اتصال به بک‌اند و Prerender بدون بلاک upstream	باز و بسته شدن هزاران سوکت TCP در ثانیه، پر شدن جدول TIME_WAIT	تعریف upstream duta_backend { keepalive 64; } و upstream duta_prerender {	استفاده مجدد از سوکت‌های باز TCP (Connecti

		)استفاده پیش‌فرض از HTTP/1.0و بستن سوکت در هر درخواست.(	هسته لینوکس در ترافیک بالا و بروز خطای 502 Bad Gateway.	keepalive 16; }همراه با proxy_http_version 1.1.	on (Pooling، حذف سربار Handshake و کاهش چشمگیر مصرف CPUشبکه.
INF-SCALE-03	nginx.conf (open_file_cache & proxy_buffers)	خواندن فایل‌های استاتیک از دیسک در هر درخواست و بافر پیش‌فرض کوچک برای پاسخ‌های پروکسی.	درگیر شدن دیسک سرور هنگام ارسال پاسخ‌های بزرگ (مانند سایت‌مپ ۷۸۵ کیلوبایتی) به دلیل نوشتن در proxy_temp_path.	فعال‌سازی sendfile on، tcp_nopush، open_file_cache)کش ۱۰۰۰ فایل در RAM)و تنظیم proxy_buffers 16 32k.	سرو کامل فایل‌های استاتیک و پاسخ‌های APIاز داخل RAMبدون کوچک‌ترین درگیری دیسک (Zero Disk I/O).